

Static Pixels Are Not Static Tokens: A Pre-Registered Study of VLA Acceleration on Transparent Lab Objects

Yogya Mehrotra

yogyamehrotra@gmail.com
github.com/yogyam/clearvla

Abstract

Vision-language-action (VLA) policies are being accelerated with visual-token pruning, cross-observation token caching and low-bit weight quantization, and evaluated almost exclusively on opaque tabletop objects. Laboratory manipulation is dominated by glass, whose thin, low-texture, slowly changing appearance is exactly what these methods discard. We pre-registered the hypothesis that the accelerations cost disproportionately more task success on transparent objects, and built a paired simulated lab testbed to test it: the same seed yields identical scenes with an opaque or a glass test tube (MuJoCo physics, Blender Cycles refraction), three tasks, a fully instrumented 30M-parameter VLA trained on 2,700 scripted demonstrations, and four accelerations implemented on identical weights (FastV-style pruning, VLA-Cache-style pixel-change K/V reuse, INT4 weights, and a ground-truth “protect” oracle). Across 3,400 paired closed-loop episodes with the final instrument, offline recall of the object’s patches, and latency on an NVIDIA L4 and an Apple M5 Pro, the transparency hypotheses are not supported: at every usable budget no method loses more success on glass than on opaque, quantization is material-neutral within 2 points, and the one significant material effect, at a 12.5% budget, runs the other way (opaque grasp −28 points, glass unchanged). What the study establishes is material-independent. Attention-ranked pruning keeps the object’s patches at chance level (24.5% recall at a 25% budget) and yet costs almost nothing down to that budget, because the object’s information has left the visual tokens by layer 2. Pixel-change caching is catastrophic (0% grasp and pour at 50% and 25% budgets, both materials) because patches whose pixels do not change still have ViT tokens that do (cosine 0.81 to the previous frame); a feature-change criterion fails identically. And none of the three methods reduces wall-clock latency on either device.

Data, model, and every episode log are released.

1 Motivation

Efficiency work on VLAs such as OpenVLA and π_0 (Kim et al., 2024; Black et al., 2025) reports success retained under visual-token pruning (Liu et al., 2025; Cheng et al., 2026; Yang et al., 2025), token caching (Xu et al., 2025; Wei et al., 2026) and weight quantization (Zhang et al., 2026; Wang et al., 2026; Akbari et al., 2026; Yan et al., 2026) on LIBERO (Liu et al., 2023), SIMPLER (Li et al., 2024) and similar suites, where objects are opaque, textured and well separated from the background (Yu et al., 2025). Laboratory benchmarks (Lan et al., 2025; Li et al., 2025; Ren et al., 2026) are full of glassware and name transparent liquids as an open problem, but do not evaluate acceleration. Two adjacent observations made a specific failure plausible. Token-pruning papers note that what gets dropped is “low-texture edges and narrow contact regions” (Cheng et al., 2026); a glass tube is both. And VLA-Cache’s own ablation shows that naive reuse of pixel-static tokens costs about ten points of success because task-relevant regions that barely change get cached (Xu et al., 2025); a glass tube changes few pixels as the arm approaches it. If the accelerations discard exactly the evidence a transparent object provides, laboratory deployments would inherit a failure mode that opaque-object benchmarks cannot reveal.

We tested this as a pre-registered study (Nosek et al., 2018): the hypotheses, metrics and decision rules in Table 1 were fixed before any model was trained, and the closed-loop design was gated on the cheap offline hypothesis H1. The result is a negative one on the question asked and a positive one on three questions it forced open.

Contributions. (1) A paired opaque/glass simulated lab testbed with three tasks and a from-

Hypothesis	Supported if
H1: importance scores under-rank transparent regions	recall of the object’s patches at the 25% budget: opaque – glass ≥ 10 pts, bootstrap CI excludes 0
H2 (main): speed-ups cost more success on glass	at 25%, retained success ≥ 15 pts lower on glass, CI excludes 0, on ≥ 2 of 3 tasks
H3: each method fails where its assumption fails	Cache hurts pour most; Prune hurts grasp most
H4: quantization is material-neutral	glass – opaque difference < 5 pts
H5: protecting instruction-referenced patches recovers the loss	$\geq 50\%$ of the transparency-specific drop recovered at $\leq 10\%$ extra compute

Table 1: Pre-registered hypotheses. Decision gate: if H1 fails, the closed-loop matrix runs at a single budget with 50 paired episodes per cell instead of the full 5,400-episode design.

scratch 30M-parameter VLA whose attention maps and key/value caches are fully exposed, so that accelerations can be implemented on identical weights and inspected. (2) Offline and closed-loop evidence, over 3,400 paired episodes with the final instrument (6,300 across all instrument versions), that the transparency hypotheses fail at every usable budget, with a power statement. (3) Three material-independent mechanism findings: pruning keeps the task object at chance yet costs nothing to a 25% budget; pixel-static patches have non-static ViT tokens, which makes cross-observation K/V reuse catastrophic; and none of the methods moves latency on an L4 or an M5 Pro. (4) A reversed material effect at a 12.5% budget, reported as an unreplicated observation. (5) The dataset, checkpoint and every episode log, released.

2 Testbed

Scenes. Scenes are built per seed with MuJoCo 3.13 (Todorov et al., 2012) and the Menagerie Franka Panda (Zakka et al., 2022): a bench, a test tube (radius 14 mm, height 100 mm) and one of a rack, a holder or a beaker with an inner fill mark. Per-seed randomisation covers object position (± 5 cm), receptacle position (± 3 cm), light intensity ($\pm 20\%$) and camera pose (± 1 cm, $\pm 2^\circ$). The same seed under `opaque` and `glass` gives identical geometry, expert actions and segmentation masks; only the material differs, so every comparison in the paper is paired.

Rendering. Frames are rendered by Blender 5.2 Cycles (Blender Foundation, 2026) driven in-process, with the geometry rebuilt from MuJoCo’s tables each episode. Glass uses a Cycles glass BSDF with transparent shadow rays, so the tube appears as faint refraction and a soft shadow rather than a silhouette (Figure 2). Two 224^2 views are used: a fixed diagonal camera and a wrist camera looking down the tool axis. Segmentation masks of the instruction-referenced objects come from MuJoCo’s renderer and are pixel-aligned with the Blender frames.

Tasks. Control runs at 10 Hz for 300 steps; success must hold for 2 s. *Grasp-and-rack*: place the tube within 1 cm of the slot, upright, released. *Pour-to-line*: a scripted liquid model (fill rate proportional to tilt past 55° ; liquid leaves from the lowest point of the mouth’s rim; any pour outside the beaker’s inner wall is a spill); success is a fill within $\pm 10\%$ of the mark with no spill. *Insert*: the tube starts in the gripper and must be seated in a 30 mm slot with 1 mm clearance. Instructions are eight material-neutral paraphrases per task (“put the tube in the rack”). A weld constraint attaches the tube when the gripper closes on it and releases it on open, because the Menagerie finger pads let a tilted tube wedge out during a 10 s pour; this removes grasp-slip from a perception study and is the device several LeRobot simulation tasks use (Cadene et al., 2024).

Demonstrations. Scripted waypoint experts (99–100% success on 100 seeds per cell, both materials) recorded 300 clean episodes per task and material plus 150 recovery episodes per cell in the DART style (Laskey et al., 2017): 6 mm execution noise with clean labels. Every fourth control step was rendered in both views: 2,700 episodes and 375k frames.

Instrument. Model A is a small VLA in the shape of π_0 and SmolVLA (Black et al., 2025; Shukor et al., 2025), chosen so that the acceleration hooks are explicit: a frozen SigLIP ViT-B/16 (Zhai et al., 2023) on each view (392 visual tokens) and a frozen SigLIP text encoder (64 tokens), a 19-D proprioceptive token, a 6-layer $d=512$ prefix transformer, and a 4-layer $d=384$ flow-matching action expert that cross-attends to the prefix output and predicts a 16-step chunk of 10-D actions (position relative to the current tool centre point, 6-D rotation, gripper). It has 30.4M trainable pa-

rameters, was trained for 35k steps (9 h on an M5 Pro), and runs 10 Euler steps at inference, executing 8 of 16 actions before re-observing. The prefix runs once per observation and exposes every attention map and key/value. Two design changes were required before the model was usable as an instrument, both found by closed-loop tracing: the wrist camera, and actions relative to the current tool position rather than absolute (a 10-pixel tube is smaller than a SigLIP patch; relative targets raised grasp success from 35% to 80%). Appendix B lists the versions.

Validity gate. On 100 unseen paired seeds per cell the instrument reaches grasp 77 / 80%, insert 98 / 99% and pour 43 / 37% (opaque / glass). Pour is below the 60% bar the design set; its failures are spills at the spout, and the opaque beaker hides its fill level from every viewpoint but straight down, which is the study’s only material effect on the base policy. We accepted pour as a weaker instrument rather than spend a fourth training cycle and report it with that caveat.

SmolVLA. A fine-tuned SmolVLA (Shukor et al., 2025) on the same data reached grasp 53 / 48%, pour 7 / 10% and insert 52 / 50%, fitting the demonstrations to 2 mm offline while approaching 3 cm off the tube closed-loop. It was not used further; Appendix A reports it as a negative result.

3 Acceleration methods

All four methods run inside Model A on identical weights. A budget b is the fraction of the 392 visual tokens (both views, ranked jointly) that receives full prefix compute after layer 2; layers 0–2, the text and proprioceptive tokens, the frozen encoders and the action expert are never touched. This bounds the achievable FLOPs saving at 37% ($b=12.5\%$), which we state rather than hide.

Prune follows FastV (Chen et al., 2024): at layer 2, each visual token is scored by the attention it receives from the text and proprioceptive queries, and the top- b are kept for layers 3–6. **Cache** follows VLA-Cache (Xu et al., 2025): each 16×16 pixel patch is compared with the same patch in the previous observation (cosine similarity of raw pixels); the b most-changed tokens are recomputed through layers 3–6 while the rest reuse the cached key/value pairs and layer outputs; text and proprioceptive tokens are always recomputed; the first observation of an episode is com-

At 25% budget	Opaque	Glass	Gap [95% CI]
Prune	24.5	21.5	+3.0 [+1.8, +4.2]
Cache	66.8	70.7	−3.9 [−4.5, −3.4]

Table 2: Critical-patch recall (%) at the pre-registered 25% budget, pooled over tasks and views; gap = opaque − glass with a paired bootstrap over seeds (10k resamples).

puted in full. The pixel-change criterion is the assumption under test; a *feature-change* variant ranks tokens by the change of their SigLIP feature vector instead. **Quant** applies fake INT4 weights, symmetric per group of 64 input channels, to all 84 linear layers of the prefix and expert (8.8% relative weight error), with fp16 activations. **Protect** (the H5 oracle) is Prune or Cache with tokens that overlap the simulator’s segmentation of the instruction-referenced object ($\geq 25\%$ of the patch) always kept; it is an upper bound by construction.

The 100% variants of Prune and Cache reproduce the full model bit for bit; a static scene gives zero cache drift; action drift against the full model on demonstration observations is 0.5 / 0.9 / 1.2 mm median for Prune at 50 / 25 / 12.5%, 0.8 mm for INT4, and 4.4 mm median (p90 22–26 mm) for Cache. Cost is counted analytically from the token counts each method actually uses, and latency is measured (§4.5).

4 Results

4.1 Importance scores under-rank everything, not glass

The offline set is the scripted expert driving the 50 unseen seeds 1000–1049 of every task under both materials, with both views rendered at every observation point a policy would see: 2,623 observations per material, identical frames for every method and identical trajectories across materials because the expert is state-driven. A patch is *critical* if at least 25% of its pixels lie on the instruction-referenced object; recall is the fraction of critical patches in the keep set (Figure 1, Table 2).

H1 is not supported: the gap is +3.0 points against a bar of 10. The more informative number is the level. Prune keeps 24.5% of the object’s patches at a 25% budget, which is chance; only 0.6–4.7% of layer-2 attention from the text and proprioceptive queries lands on the object at all. The glass tube receives about half the attention

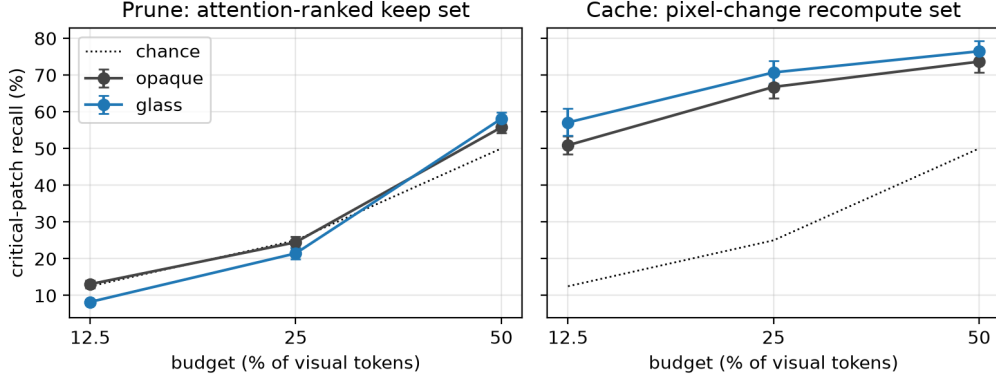


Figure 1: Critical-patch recall of the keep set vs. budget (50 unseen paired seeds per task, 2,623 observations per material, 95% bootstrap CIs over episodes). Prune (left) sits on the chance line for both materials; Cache (right) is above chance and higher for glass.

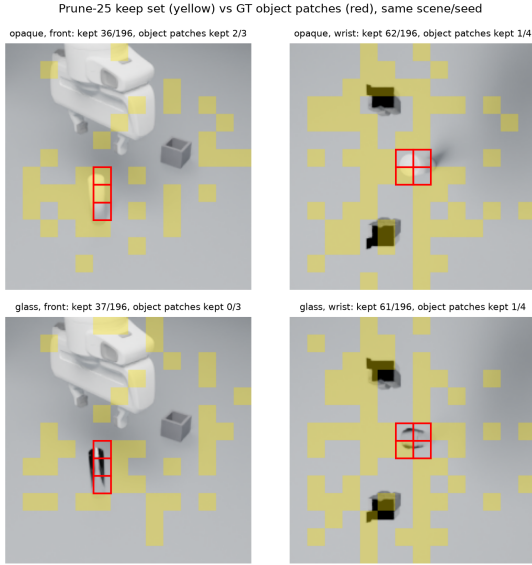


Figure 2: The same scene and seed under both materials, both views; Prune-25 keep set in yellow, ground-truth object patches outlined in red. The opaque front view keeps 2 of 3 object patches, the glass front view 0 of 3; the wrist view keeps 1 of 4 in both.

mass of the opaque one (2.1% vs. 4.7% on grasp), which is the effect the hypothesis predicted, but it is small and concentrated in grasp (+8.0 points at 25%, +9.7 at 12.5%). Cache goes the other way: the glass tube refracts the moving arm and background, so its patches change *more* pixels between observations and are recomputed more often (on pour at 12.5%: 70% opaque vs. 88% glass). The premise that glass changes pixels less than it changes state is not what the renders show.

4.2 Speed-ups cost the same on glass

The pre-registered matrix (Full, Prune-25, Cache-25, Quant, plus the feature-change cache and the Protect oracle) ran on 50 paired seeds per cell, 1,800 episodes; three further budgets added 900 (Table 3, Figure 3). The paired material difference-in-differences are in Table 4.

H2 is not supported. At the 25% budget no method loses more on glass than on opaque; every CI includes zero. With $n=50$ per cell the study excludes glass-specific losses of roughly 20 points or more, not smaller ones. **H3** has the pre-registered direction for Prune, whose largest loss is on grasp ($-8 / -4$ points), but the loss is small; Cache floors every task on both materials, so its task ordering is uninformative. **H4 is supported:** INT4 costs the same on both materials within 2 points. (The pre-registered ratio metric, retained success, gives +12 on pour with a CI of $[-75, +90]$, an artefact of dividing by a 28% base rate; absolute differences are reported alongside it in Appendix C.) **H5 is not applicable:** there is no transparency-specific loss to recover, and the oracle that keeps 100% of the object’s patches at the same budget lands within noise of plain Prune-25 on every cell, which is itself informative (§7).

4.3 A reversal at 12.5%

The one significant material effect in the study appears at the most aggressive budget and runs opposite to H2: opaque grasp falls by 28 points while glass does not move (DiD $-28 [-52, -4]$), and glass pour rises. This is one budget, $n=50$, and one of 18 comparisons, so we report it as an observation. It is consistent with the H1 numbers:

Configuration	Grasp		Pour		Insert	
	Opaque	Glass	Opaque	Glass	Opaque	Glass
Full	84	82	42	28	92	100
Prune-50	84	78	28	20	98	100
Prune-25	76	78	34	32	94	100
Prune-12.5	56	82	40	50	92	96
Protect-prune-25 (oracle)	80	76	46	26	90	96
Cache-50 (pixel)	0	2	0	0	48	66
Cache-25 (pixel)	0	0	0	0	22	24
Cache-25 (feature)	0	0	0	0	12	20
Quant INT4	76	74	38	22	94	100

Table 3: Closed-loop success (%) on 50 paired unseen seeds per cell (seeds 1000–1049; synchronous evaluation; horizon 300). Per-cell 95% CIs span about ± 14 points; full tables with CIs are in Appendix C.

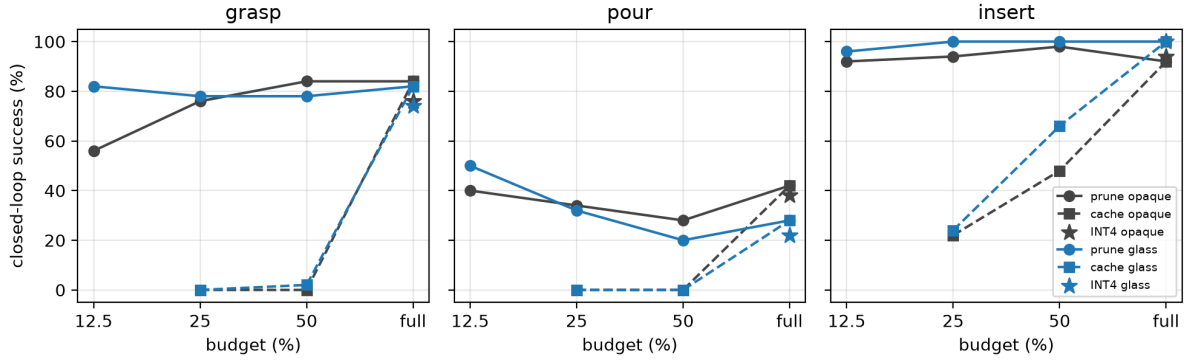


Figure 3: Closed-loop success vs. budget per task; circles Prune, squares Cache, stars INT4; black opaque, blue glass. Prune is flat to 25% on every task and both materials; Cache is at the floor at 50% and 25%; INT4 tracks Full.

Config	Task	DiD [95% CI]
Prune-25	grasp	−4 [−26, +18]
	pour	−12 [−34, +8]
	insert	+2 [−6, +10]
Prune-12.5	grasp	−28 [−52, −4]
	pour	−24 [−48, 0]
	insert	+4 [−6, +16]
Cache-25	pour	−14 [−32, +4]
Quant INT4	grasp	0 [−18, +18]
	pour	+2 [−26, +28]
	insert	+2 [−8, +12]

Table 4: Material difference-in-differences: (success − Full) on opaque minus the same on glass, in points, paired over seeds with a 10k bootstrap. A negative value means glass lost *less*. Rows not shown lie within ± 6 with CIs spanning zero.

at 12.5% the keep set retains 13% of the opaque tube’s patches and 8% of the glass tube’s, yet only the opaque policy suffers, so success on glass at this budget cannot depend on the tube’s own patches. A plausible reading is that the glass tube’s evidence, its shadow, its refraction and the background it distorts, is spread over more of the image

than an opaque silhouette and survives chance-like pruning better; we did not test this.

4.4 Static pixels are not static tokens

Cache scores 0% on grasp and pour at both 50% and 25% budgets, on both materials, and nine of ten failures knock the tube over. Figure 4 shows why. A ViT token depends on the whole image, not on its own pixels: among patches whose pixels are static between consecutive observations (similarity above 0.995, 13% of patches), the SigLIP token’s cosine to its previous value is 0.81 on average and below 0.55 for a tenth of them. Reusing key/value pairs for pixel-static tokens therefore feeds the prefix stale features on every frame in which anything moved, which in manipulation is every frame. The drift accumulates along the approach (1 to 23 mm over the first six observations at a 50% budget), the arm arrives 2–3 cm off, and the tube is knocked over. The feature-change variant fails identically (Table 3), so it is the reuse, not the pixel criterion, that breaks the policy. VLA-Cache’s published version filters reuse by atten-

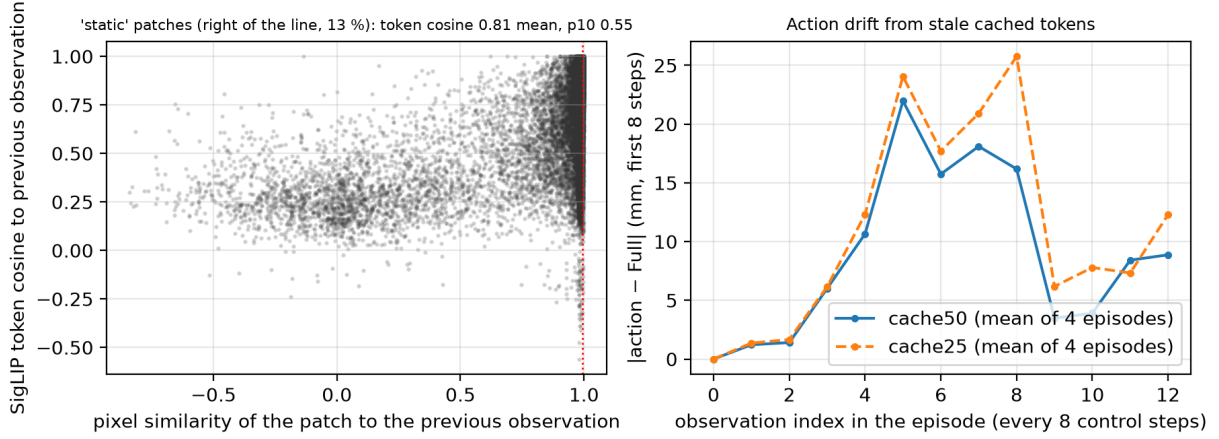


Figure 4: Left: pixel similarity of a patch to the previous observation vs. the cosine of its SigLIP token to the previous observation, over consecutive expert observations; 13% of patches are pixel-static (right of the dotted line) and their tokens still move (cosine 0.81 mean, tenth percentile 0.55). Right: action drift of Cache-50 and Cache-25 against the full model along expert episodes, mean of four grasp episodes.

tion and reports about ten points of loss from naive reuse on its benchmark (Xu et al., 2025); for a policy that depends on a 10-pixel object, naive reuse is not a ten-point loss but a total one. Transparency never enters.

4.5 Nothing is faster

Table 5 measures the accelerated region (prefix and 10-step expert) at batch 1. Eager execution on the L4 is launch-bound: 65 ms whatever the token count. Removing launch overhead with CUDA graphs or TensorRT is a $12\times$ speed-up by itself; after that, a 37% FLOPs reduction from pruning buys 14% of latency under CUDA graphs and nothing under TensorRT, whose per-layer fixed costs dominate at these sizes. On the M5 Pro the frozen vision encoder is 55% of the observation time and the ten-step expert 37%; the prefix the methods touch is 9%, and pruning or caching add gather overhead rather than save time. Fake INT4 changes nothing by construction; a real INT4 kernel would save weight bandwidth, which is not the bottleneck for a 30M-parameter model at batch 1. For a policy of this size, none of the three methods is a latency lever, on either device.

5 Related Work

Token pruning for VLAs. FastV (Chen et al., 2024) showed that a vision-language model can drop half its visual tokens after layer 2 with little loss, and VLA variants (Liu et al., 2025; Cheng et al., 2026; Yang et al., 2025) report success retained on LIBERO and SIMPLER at 25–50% of

tokens. VLA-IAP (Cheng et al., 2026) measures retention of gripper-object boundary tokens, the closest mechanistic argument to H1, but never varies material. Our finding that the object’s patches are kept at chance yet success survives (§4.1, §4.2) suggests that boundary-token retention is not what makes pruning work in a small trained-from-scratch policy.

Token caching. VLA-Cache (Xu et al., 2025) reuses key/value pairs of pixel-static tokens with an attention-based relevance filter, and its ablation of naive reuse is the closest published point to §4.4; a learned successor (Wei et al., 2026) decides which tokens to cache. Neither reports the token-level drift of pixel-static patches under a ViT encoder that we measure.

Quantization and compression fragility. Post-training quantization of VLAs to W4A4 and below (Zhang et al., 2026; Wang et al., 2026; Akbari et al., 2026; Yan et al., 2026) reports closed-loop error compounding at low bit widths; our H4 result agrees that 4-bit weights are benign for a small model and adds that the cost is material-neutral. Jabbour et al. (2025) find the vision backbone far more fragile than the language model under weight pruning; our observation that the object’s information leaves the visual tokens by layer 2 is a token-side counterpart.

Lab benchmarks and transparency. AutoBio (Lan et al., 2025), LabUtopia (Li et al., 2025) and LabVLA (Ren et al., 2026) build laboratory simulators and benchmarks with glassware and name

NVIDIA L4, budget	Tokens	GFLOPs	Eager	CUDA graphs	TensorRT
Full	457	24.4	64.8	5.44	5.51
Prune-50	261	19.0	65.3	5.17	5.72
Prune-25	163	16.5	64.1	4.88	5.82
Prune-12.5	114	15.3	64.1	4.67	6.05
SigLIP, two views	–	≈ 67	7.3	2.5	–

Apple M5 Pro, config	SigLIP	Prefix	Expert	Total
Full	27.8	4.5	18.9	51.4
Prune-25	28.6	5.8	20.2	54.6
Prune-12.5	28.6	5.8	20.6	55.0
Cache-25	28.7	5.6	20.8	54.9
Quant INT4 (fake)	28.8	4.8	20.7	54.2

Table 5: Latency per observation in ms (batch 1, median of 300 runs). Top: NVIDIA L4, prefix + 10 expert steps; tokens = tokens after layer 2; GFLOPs analytic for prefix + expert; eager fp16 autocast, `torch.compile` with CUDA graphs, and TensorRT fp16. Bottom: Apple M5 Pro, eager PyTorch on MPS, by stage (SigLIP on two views; expert = 10 Euler steps).

transparent liquids as a gap, without acceleration; SEVO (Fang et al., 2026) grasps transparent bottles with active illumination and no opaque control. Robustness suites (Morgan et al., 2026; Fei et al., 2025) vary colour, texture, lighting and distractors but not transparency and not accelerated models. To our knowledge no prior work evaluates an acceleration method as a function of object transparency; this study does, and finds the effect absent at the budgets where the methods are usable.

6 Limitations

- **Simulated glass is easier than real glass.** Cycles renders clean refraction with no sensor noise, no specular highlights from laboratory lighting and no depth-sensor failure. A real-glass replication could reveal effects the simulator cannot.
- **The liquid is scripted, not simulated,** and the pour instrument is below the 60% validity bar (43 / 37%). Pour results carry wider effective uncertainty than their CIs suggest.
- **Power.** $n=50$ per cell bounds the detectable glass-specific loss at about 20 points. Smaller effects, including the +3-point H1 gap and the +8-point grasp gap, are real but below the pre-registered bar.
- **One model, one seed, one data regime.** The reversed effect at 12.5% is unreplicated and is one of 18 comparisons.
- **Bounded saving by design.** Layers 0–2, the

encoders and the expert are untouched, so the achievable FLOPs reduction is 37%; a method that prunes before the encoder would face the object-localisation problem this design side-steps.

- **Oracle masks and synchronous evaluation.** Protect and the H1 masks use simulator segmentation; the simulator pauses during inference, so latency cannot affect success.
- **The sticky-gripper weld** removes grasp-slip as a failure mode, as in several LeRobot simulation tasks.
- **SmolVLA was not diagnosed to the root.** The negative result in Appendix A identifies the closed-loop failure but not which of the frozen VLM, the 64-token image budget or the per-frame action convention causes it.

7 Implications

For someone about to accelerate a laboratory VLA, the study replaces one worry with two. Material is not the variable to test first: at every budget where pruning and quantization keep the policy usable, glass and opaque objects lose the same. The variables to test are the interaction between the vision encoder and any cross-observation cache, and the latency budget itself. On the first, a frozen ViT makes every token a function of the whole image, so “static” is a property of tokens, not of pixels, and a cache must be validated by token drift, not pixel change; the feature-change variant shows that even a token-level criterion does not rescue reuse when three quarters of the tokens stay stale.

On the second, a 30M-parameter policy at batch 1 spends its time in the encoder, the iterative expert and kernel launches, none of which token methods touch; the honest speed-up in Table 5 came from compilation, not from any of the three methods.

Two methodological points follow. Recall of the task object’s patches, the quantity H1 was built on and the one that pruning papers often argue from, did not predict closed-loop success here: the object’s information had already left the visual tokens by the layer at which pruning acts, and an oracle that kept every object patch changed nothing. And pre-registration is what makes the negative result usable: the hypotheses, the bar and the decision gate were fixed in advance, the gate was triggered, and the three mechanism findings were not what the study was looking for.

Reproducibility

All code, the pre-registered plan (PLAN.md), the weekly logs, every closed-loop episode (6,300 JSONL records across all instrument versions, 3,400 with the final one, with per-observation keep sets, object-patch recall and FLOPs) and the analysis scripts are at <https://github.com/yogyam/clearvla>; scripts/reproduce.md maps every table and figure to one command, and the analysis-only path runs in minutes without a simulator. The dataset (LeRobot v3, 2,700 episodes, both views, CC BY 4.0) is at <https://huggingface.co/datasets/yogyamehrotra/clearvla-sim> and the Model A v4 checkpoint at <https://huggingface.co/yogyamehrotra/clearvla-model-a-v4>. Total compute: about 60 h of M5 Pro time and \$9 of cloud GPU.

References

- Arash Akbari, Arman Akbari, Masih Eskandar, Qitao Tan, et al. 2026. ActQuant: Sub-4-bit action-guided quantization for vision-language-action models. *arXiv preprint arXiv:2605.24011*.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, et al. 2025. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*. ArXiv:2410.24164.
- Blender Foundation. 2026. Blender 5.2 and the Cycles renderer. <https://www.blender.org>.
- Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, and Thomas Wolf. 2024. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. <https://github.com/huggingface/lerobot>.
- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision (ECCV)*. ArXiv:2403.06764.
- Jintao Cheng, Weibin Li, Haozhe Wang, Gang Wang, et al. 2026. Training-free interaction-aligned visual token pruning for efficient embodied manipulation. *arXiv preprint arXiv:2603.22991*.
- Tianchonghui Fang, Yuan Zhuang, and Fei Miao. 2026. SEVO: Semantic-enhanced virtual observation for robust VLA manipulation via active illumination and data-centric collection. *arXiv preprint arXiv:2605.11114*.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, et al. 2025. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*.
- Jason Jabbour, Dong-Ki Kim, Max Smith, Jay Patrikar, et al. 2025. Don’t run with scissors: Pruning breaks VLA models but they can be recovered. *arXiv preprint arXiv:2510.08464*.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, et al. 2024. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*. ArXiv:2406.09246.
- Zhiqian Lan, Yuxuan Jiang, Ruiqi Wang, Xuanbing Xie, et al. 2025. AutoBio: A simulation and benchmark for robotic automation in digital biology laboratory. *arXiv preprint arXiv:2505.14030*.
- Michael Laskey, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg. 2017. DART: Noise injection for robust imitation learning. In *Conference on Robot Learning (CoRL)*. ArXiv:1703.09327.
- Rui Li, Zixuan Hu, Wenxi Qu, Jinouwen Zhang, et al. 2025. LabUtopia: High-fidelity simulation and hierarchical benchmark for scientific embodied agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2505.22634.
- Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, et al. 2024. Evaluating real-world robot manipulation policies in simulation. In *Conference on Robot Learning (CoRL)*. ArXiv:2405.05941.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2306.03310.

- Ziyan Liu, Yeqiu Chen, Hongyi Cai, Tao Lin, et al. 2025. Bridging the semantic-action gap in visual token pruning for efficient VLA inference. *arXiv preprint arXiv:2511.16449*.
- Jeremy Morgan, Hyeonho Oh, Prajwal Vijay, Jincen Song, Ashvin Arora, et al. 2026. Colosseum V2: Benchmarking generalization for vision-language-action models. *IEEE Robotics and Automation Letters*. ArXiv:2605.27759.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. 2018. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606.
- Baochang Ren, Xinjie Liu, Xi Chen, Yanshuo Liu, et al. 2026. LabVLA: Grounding vision-language-action models in scientific laboratories. *arXiv preprint arXiv:2606.13578*.
- Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, et al. 2025. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- Xinyu Wang, Mingze Li, Sicheng Lyu, Dongxiu Liu, et al. 2026. HoloQ-VLA: Uniform W4A4 quantization of vision-language-action models. *arXiv preprint arXiv:2605.28803*.
- Yujie Wei, Jiahao Fan, Jiyu Guo, Ruichen Zhen, et al. 2026. Learning to accelerate vision-language-action models through adaptive visual token caching. *arXiv preprint arXiv:2602.00686*.
- Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. 2025. VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2502.02175.
- Xin Yan, Zhenglin Wan, Feiyang Ye, Xingrui Yu, Hangyu Du, Yang You, and Ivor Tsang. 2026. HBVLA: Pushing 1-bit post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.13710*.
- Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. 2025. EfficientVLA: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*.
- Zhaoshu Yu, Bo Wang, Pengpeng Zeng, Haonan Zhang, et al. 2025. A survey on efficient vision-language-action models. *arXiv preprint arXiv:2510.24795*.
- Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. 2022. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. https://github.com/google-deepmind/mujoco_menagerie.
- Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. 2023. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*. ArXiv:2303.15343.
- Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, et al. 2026. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. ArXiv:2602.20309.

A SmolVLA as a second instrument

lerobot/smolvla_base (Shukor et al., 2025; Cadene et al., 2024) was fine-tuned on the same 2,700 episodes, with our cameras mapped to its camera1/camera2 slots, 224² frames padded to 512², and actions as per-frame deltas because the LeRobot format stores one action per frame; 20k steps at batch 64 on an H100 (101 min), with only the 100M-parameter action expert trainable (the library default). On 100 unseen paired seeds per cell it reached grasp 53 / 48%, pour 7 / 10% and insert 52 / 50% (opaque / glass), against 77 / 80, 43 / 37 and 98 / 99 for Model A v4. Offline it reproduces the demonstrations to 2.0 / 1.9 / 2.6 mm per axis with correct gripper timing; closed-loop it descends 3 cm to the side of the tube without correcting laterally. Re-observing every 4 or 2 steps instead of 8 did not help (7 / 20 vs. 6 / 20 grasp on the same seeds). We attribute the gap to the frozen web-pretrained VLM (64 visual tokens per 512² image) and the per-frame action convention, and did not spend further budget on it.

B Instrument versions

v1: front camera only, absolute-pose actions, 150 demonstrations per cell; grasp 39 / 33%, pour 8 / 7%. v2: added the wrist camera and 150 demonstrations per cell; no closed-loop improvement despite 3–6 mm offline error, and a trace showed the gripper closing 23–31 mm off the tube. v3: actions relative to the current tool position; grasp 80 / 82%, insert 98 / 96%, pour 10 / 22%. v4: 150 DART recovery demonstrations per cell; grasp 77 / 80%, pour 43 / 37%, insert 98 / 99% (the instrument used throughout). The offline error of v2 (3–

6 mm) and its closed-loop failure are the same signature later seen in SmolVLA.

C Full tables

Per-cell 95% bootstrap CIs, the pre-registered ratio-based diff-in-difference, failure-mode histograms, per-view and per-task H1 recall, and the budget-curve tables are in the repository: `results/week5/matrix_report.md`, `results/week6/matrix_report.md`, `results/week4/h1_report.md`. Representative CIs: Full grasp 84 [74, 94] / 82 [70, 92]; Full pour 42 [28, 56] / 28 [16, 40]; Cache-25 grasp 0 [0, 0] / 0 [0, 0]; Prune-12.5 grasp 56 [42, 70] / 82 [70, 92].